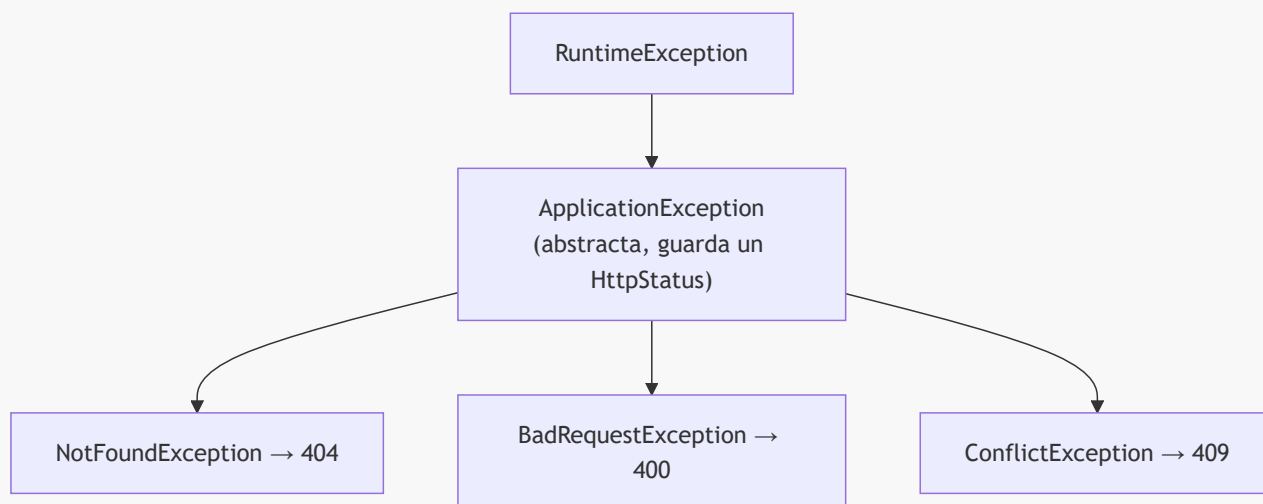


Fase 5 — Manejo de errores

Ya lo hiciste en la Práctica 7 — esta fase es sobre todo repaso activo, no contenido nuevo.

1. Excepciones custom extendiendo `RuntimeException`



```
public class ApplicationException extends RuntimeException {
    private final HttpStatus status;

    protected ApplicationException(HttpStatus status, String message) {
        super(message);
        this.status = status;
    }

    public HttpStatus getStatus() { return status; }
}
```

```
public class NotFoundException extends ApplicationException {
    public NotFoundException(String message) {
        super(HttpStatus.NOT_FOUND, message); // 404
    }
}
```

`ApplicationException` es la base: cada subclase solo fija **qué status HTTP** le corresponde. El resto del código lanza `throw new NotFoundException("Product not found")` sin pensar en HTTP — el status ya viaja pegado a la excepción.

Se extiende `RuntimeException` (no `Exception` a secas) para que sea **unchecked**: no obliga a poner `throws` ni `try/catch` en cada `Service`.

Uso real en `ProductServiceImpl`:

```
private ProductEntity findActiveProductOrThrow(Long id) {
    return repository.findById(id)
```

```

        .filter(product -> !product.isDeleted())
        .orElseThrow(() -> new NotFoundException("Product not found"));
    }

```

2. @RestControllerAdvice + @ExceptionHandler

```

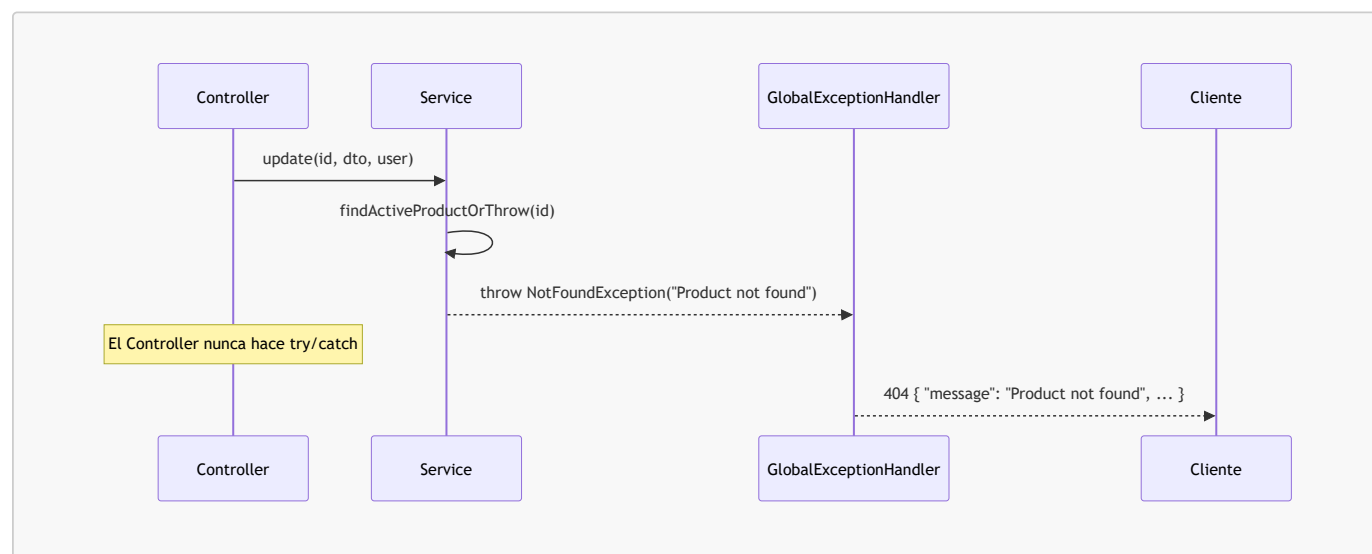
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApplicationException.class)
    public ResponseEntity<ErrorResponse> handleApplicationException(
        ApplicationException ex, HttpServletRequest request) {
        ErrorResponse response = new ErrorResponse(ex.getStatus(), ex.getMessage(),
            request.getRequestURI());
        return ResponseEntity.status(ex.getStatus()).body(response);
    }
}

```

`@RestControllerAdvice` = interceptor global para **todos** los Controllers. Cada

`@ExceptionHandler(TipoDeExcepcion.class)` es un `catch` centralizado: en vez de poner `try/catch` en cada endpoint, una excepción lanzada en cualquier Service sube hasta aquí y se convierte en una respuesta HTTP consistente.



Manejadores que ya tienes en `GlobalExceptionHandler`:

<code>@ExceptionHandler(...)</code>	Cuándo se dispara	Status
<code>ApplicationException</code>	Cualquier <code>NotFoundException</code> / <code>BadRequestException</code> / <code>ConflictException</code> propia	El que trae la excepción (404/400/409)
<code>MethodArgumentNotValidException</code>	Falla <code>@Valid</code> <code>@RequestBody</code>	400 + details por campo
<code>BindException</code>	Falla validación en <code>@ModelAttribute</code> (query params, ej. <code>PaginationDto</code>)	400 + details por campo
<code>AuthorizationDeniedException</code>	<code>@PreAuthorize</code> evalúa <code>false</code> (rol insuficiente)	403

<code>@ExceptionHandler(...)</code>	Cuándo se dispara	Status
<code>AccessDeniedException</code>	Lanzada a mano en el Service (ej. <code>validateOwnership</code>)	403
<code>AuthenticationException</code>	Token inválido/expirado	401
<code>Exception</code> (genérico, al final)	Cualquier cosa no capturada arriba	500

El orden de `@ExceptionHandler` no importa para Spring (matchea por tipo más específico), pero **conceptualmente** el genérico `Exception.class` es la red de seguridad final — si le llega algo, es porque nadie más lo esperaba.

3. Cómo devolver un 404 y un 400 con detalle de campos

`ErrorResponse` es la forma única de cuerpo de error de toda la API:

```
public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;      // "Not Found", "Bad Request"...
    private String message;
    private String path;
    private Map<String, String> details; // solo en errores de validación
}
```

404 simple (`NotFoundException`):

```
{
  "timestamp": "2026-07-19T10:00:00",
  "status": 404,
  "error": "Not Found",
  "message": "Product not found",
  "path": "/api/products/999"
}
```

400 con detalle por campo (`MethodArgumentNotValidException`, viene de `@Valid`):

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationException(
    MethodArgumentNotValidException ex, HttpServletRequest request) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
        .forEach(error -> errors.put(error.getField(), error.getDefaultMessage()));

    ErrorResponse response = new ErrorResponse(HttpStatus.BAD_REQUEST, "Datos de entrada
    inválidos",
        request.getRequestURI(), errors);
    return ResponseEntity.badRequest().body(response);
}
```

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Datos de entrada inválidos",
  "path": "/api/products/999"
}
```

```
"path": "/api/products",
"details": {
  "name": "El nombre es obligatorio",
  "price": "El precio no puede ser negativo"
}
}
```

`details` viaja como `null` cuando no aplica — `@JsonInclude(JsonInclude.Include.NON_NULL)` en `ErrorResponse` lo omite del JSON en vez de mandar `"details": null`.

Resumen / Chuleta

Pregunta	Respuesta corta
¿Por qué extender <code>RuntimeException</code> y no <code>Exception</code> ?	Para que sea <i>unchecked</i> : no obliga <code>throws/try-catch</code> en cada Service.
¿Qué hace <code>@RestControllerAdvice</code> ?	Centraliza el manejo de excepciones de todos los Controllers en una sola clase.
¿Cómo sabe cada excepción qué status devolver?	<code>ApplicationException</code> guarda un <code>HttpStatus</code> ; cada subclase lo fija en su constructor.
¿Cuándo aparece <code>details</code> en el <code>ErrorResponse</code> ?	Solo en errores de validación (<code>MethodArgumentNotValidException/BindException</code>); el resto lo omite.

Cómo estudiarlo

Lee `GlobalExceptionHandler.java` línea por línea. Por cada `@ExceptionHandler`, pregúntate: "¿qué situación real del usuario dispara esto?". Luego provócalos a propósito desde Postman.

Práctica sugerida: Práctica 7

Checklist

- ☐ Sé crear una excepción custom extendiendo `RuntimeException`
- ☐ Entiendo qué hace `@RestControllerAdvice`
- ☐ Provoqué cada error de mi `GlobalExceptionHandler` desde Postman